Automated Surface Defect Identification in Production

Final Project Report

ISE 244: AI Tools and Practice for Systems Engineering
Duc Quang Tran
Spring 2026

Executive Summary

This project examines automated surface defect identification in production using a reproducible benchmarking approach for deep learning object detection models. The problem is important for manufacturing quality control because surface defects such as scratches, inclusions, patches, pitted surfaces, rolled-in scale, and crazing can affect product reliability, customer acceptance, and downstream production cost. Manual inspection is limited by subjectivity, fatigue, and throughput constraints, while AI-based inspection requires careful validation before it can be trusted in an industrial setting.

The project uses the 2025 study by Lema, Sanchez-Gonzalez, Usamentiaga, and delaCalle as the main methodological reference. The report focuses on NEU surface defect dataset, AP50 model comparison, four-fold stratified cross-validation, and statistical testing by one-way ANOVA and Tukey HSD. The main takeaway is that AP50 rankings alone are not enough: the choice of partition and statistical significance must be considered before choosing a model for quality-control support. In the benchmark table used for this project, YOLOv5m has the highest mean AP50 at 0.765, and the competitive-model ANOVA excluding the extreme YOLOv8n-DefectDef underperformer has $p = 0.0022$ with eta-squared = 0.446.

1. Problem Definition

Surface defect detection is a quality-control problem in which images of manufactured surfaces are inspected to determine whether defects are present and where they are located. In steel manufacturing, defects can appear as scratches, inclusions, patches, pitted surfaces, rolled-in scale, and crazing. These defects matter because they can reduce appearance quality, compromise mechanical performance, increase scrap or rework, and allow nonconforming components to move further into the production system.

Traditional visual inspection depends heavily on human judgment. While human inspectors are essential for final accountability, manual inspection can be time-consuming and inconsistent under high production volume. The selected research problem is therefore not simply whether a neural network can detect defects; the deeper problem is how to compare defect detection models fairly enough that an engineer can trust the result. In small industrial datasets, a model may appear better because of a favorable train/test split rather than because it is truly more effective.

This project frames automated surface defect detection as an industrial engineering decision-support problem. The goal is to support quality-control decisions with evidence, not to replace human inspection entirely. A useful model should produce strong AP50 performance, remain stable across dataset partitions, and show statistically meaningful differences when compared with alternative models.

2. Project Objectives

- Define the production quality-control problem addressed by automated surface defect detection.
- Use the NEU surface defect benchmark as the project context because it is public, small, balanced, and widely used in the literature.
- Compare object detection model performance using AP50, the benchmark metric reported in the main reference paper.
- Evaluate whether performance changes across train/test split ratios and cross-validation folds.
- Test the observed differences between the models statistically significant using one way ANOVA and Tukey HSD.
- Provide a reproducible code repository with stored benchmark data, generated figures, statistical summaries, tests, and a mini app for explaining the workflow.

These objectives connect the project to both industrial and computational perspectives. From the ISE side, the project focuses on quality control, inspection reliability, decision support, and process risk. From the CS side, it uses object detection, cross-validation, benchmark metrics, statistical analysis, and a small interactive browser application to communicate model behavior.

3. Analysis

3.1 Dataset and task

The analysis is based on the Northeastern University surface defect dataset, commonly referred to as NEU or NEU-DET in object detection form. The dataset contains 1,800 grayscale images of steel surface defects and six defect classes: crazing, inclusion, patches, pitted surface, rolled-in scale, and scratches. This is an object detection task so the model predicts the class of the defect and the bounding box location. This differs from classification, which assigns one label to an image, and segmentation, which requires pixel-level masks.

Week 12 course concepts are reflected in the analysis design. Surface defect detection is a computer vision problem where convolutional feature hierarchies, pretrained model families, data augmentation, and fine-tuning are especially relevant because the dataset is small. The project uses detection rather than segmentation because bounding boxes are sufficient for many inspection workflows and are less expensive to label than pixel-level masks.

3.2 Benchmark metric

The primary metric is AP50, average precision at an intersection-over-union threshold of 0.50. AP50 summarizes precision-recall behavior over different confidence thresholds while requiring a predicted bounding box to overlap the ground-truth box by at least 50%. This makes AP50 appropriate for comparing object detection models without depending on one fixed confidence cutoff.

3.3 Cross-validation and statistical testing

The benchmark method uses four-fold stratified cross-validation. In each fold, 75% of the data is used for training and 25% is used for testing while preserving class distribution. This design reduces bias from a single arbitrary split and makes it possible to evaluate fold-to-fold stability. The project also evaluates train/test split sensitivity using reported AP50 values for 80/20, 75/25, and 70/30 splits.

The statistical analysis is carried out in two steps. One-way ANOVA tests the null hypothesis that the AP50 model groups differ by chance only and Tukey HSD is used to identify which pairs of models are significantly different. The repository also reports eta-squared for ANOVA, which is an estimate of the proportion of AP50 variation explained by the choice of model. This adds practical interpretation beyond reporting only a p-value.

3.4 Reproducibility workflow

The code repository contains a default analysis path that runs without the NEU image dataset or GPU hardware. It reads stored AP50 tables from CSV files, regenerates model rankings, runs ANOVA and Tukey HSD, and creates figures for the report. Optional scripts support NEU XML-to-YOLO annotation conversion, fold preparation, YOLO fold training, and run aggregation for a full experiment when the external dataset and compute resources are available.

4. Results

The analysis output identifies YOLOv5m as the top model by mean AP50. Its four fold AP50 values are 0.769, 0.769, 0.762, and 0.759, giving a mean of 0.765. Several models cluster close to the top, including YOLOv5s and YOLOv5x at 0.761, and YOLO11l at 0.759. This clustering is one reason statistical testing is necessary: small AP50 differences may not be meaningful without evidence across folds.

Rank	Model	Fold AP50 values	Mean AP50	Family
1	YOLOv5m	0.769, 0.769, 0.762, 0.759	0.765	YOLOv5
2	YOLOv5s	0.769, 0.776, 0.750, 0.747	0.761	YOLOv5
3	YOLOv5x	0.764, 0.762, 0.767, 0.751	0.761	YOLOv5
4	YOLO11l	0.762, 0.763, 0.752, 0.757	0.759	YOLO11
5	YOLOv5n	0.770, 0.767, 0.747, 0.747	0.758	YOLOv5
6	YOLOv5l	0.775, 0.767, 0.742, 0.744	0.757	YOLOv5

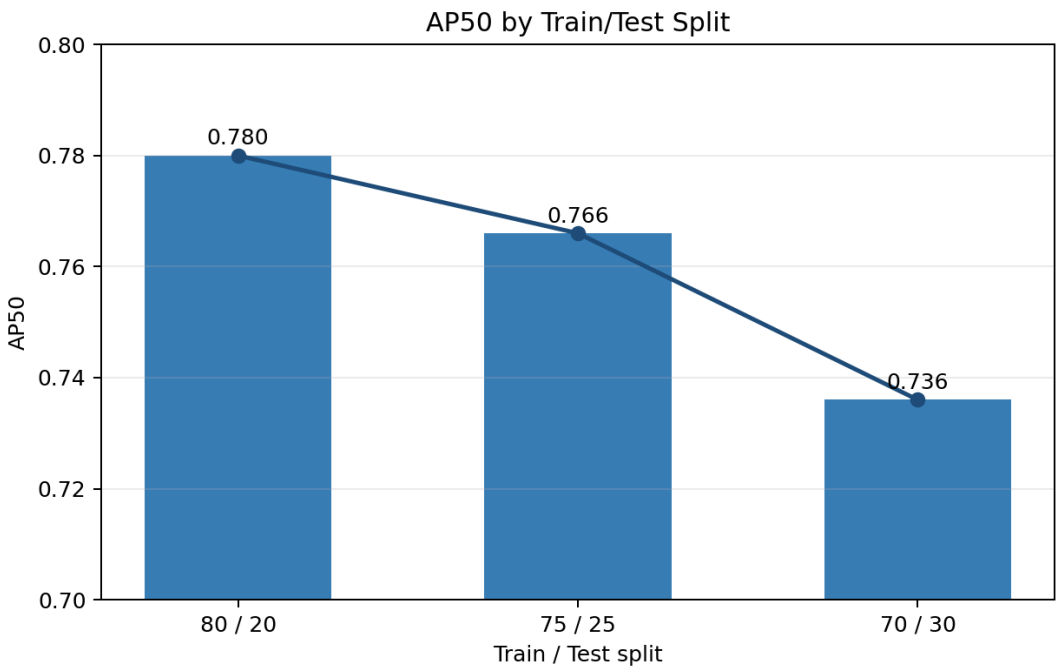

Figure 1. AP50 varies across train/test split ratios, showing why a single split can bias conclusions.

The split sensitivity result shows a decrease in AP50 from 0.780 at 80/20 train/test split to 0.736 at 70/30 split. This supports the main concern of the paper selected: the architecture alone is not the only factor that leads to the change in model performance depending on the dataset partition.

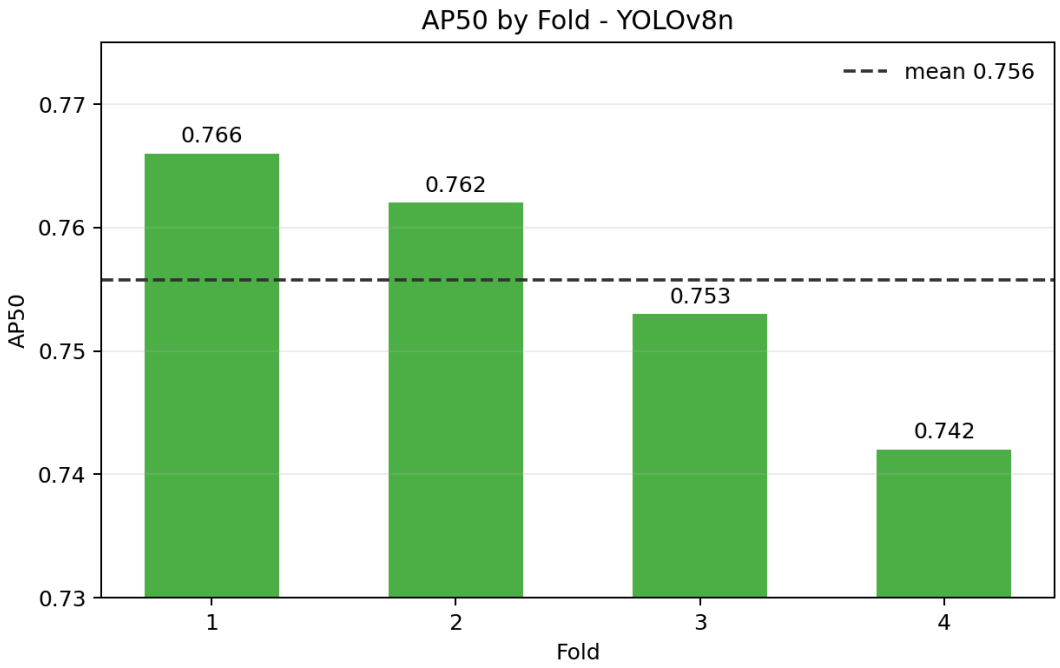

Figure 2. YOLOv8n fold-level AP50 values show variation across cross-validation folds.

The fold-level result for YOLOv8n ranges from 0.766 to 0.742. Even for a competitive model, fold-to-fold variation is visible. This means that an industrial model comparison should report both central tendency and stability, not just the best single run.

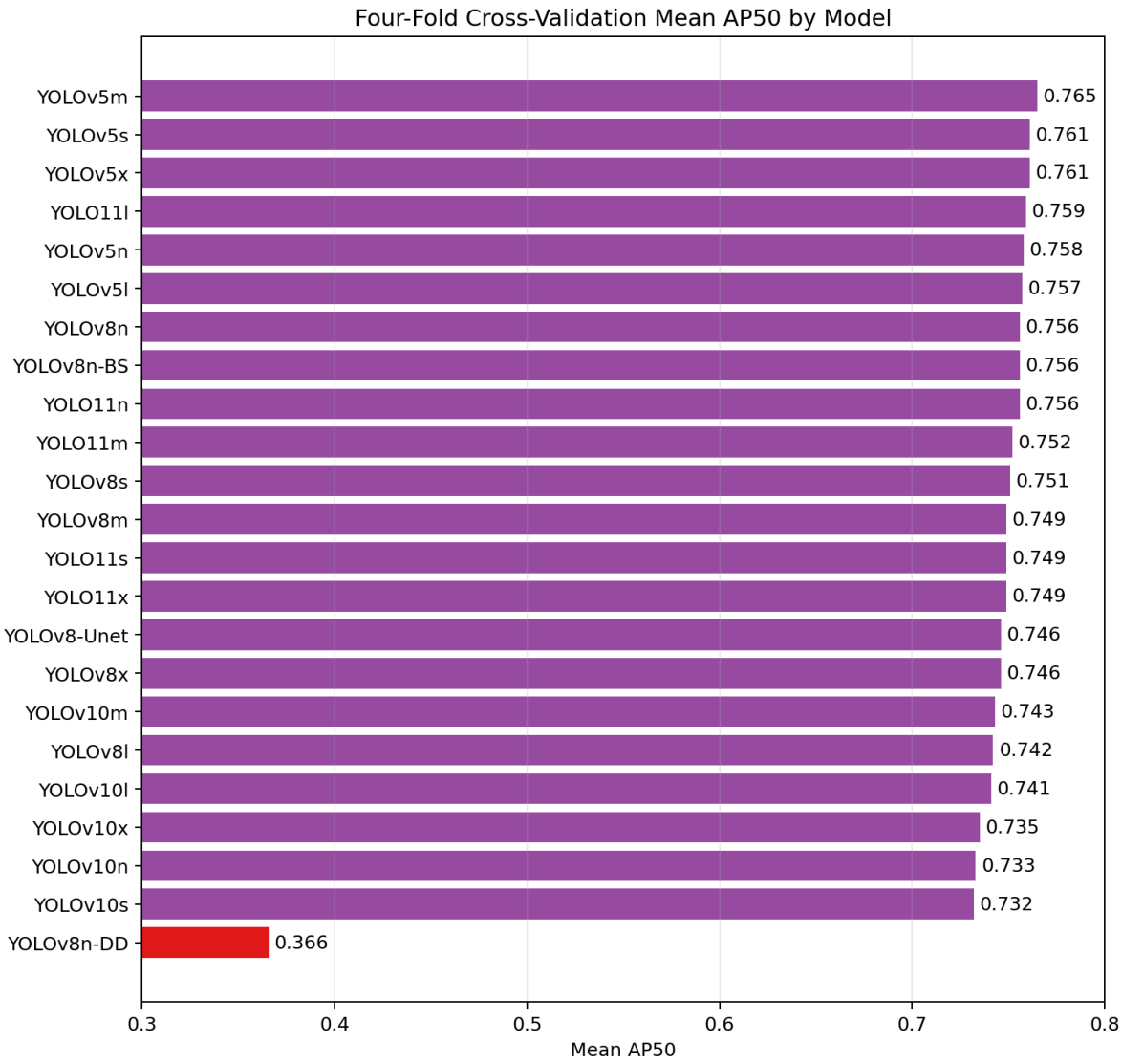

Figure 3. Mean AP50 by model across the four-fold evaluation protocol.

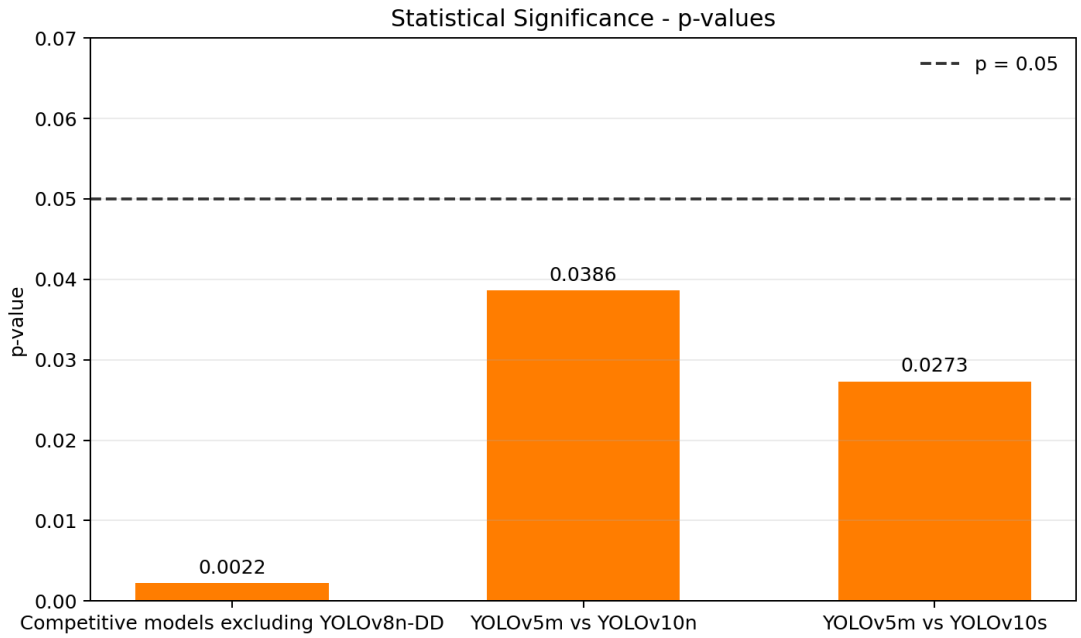

Figure 4. Statistical significance summary using ANOVA and Tukey HSD p-values.

Comparison set	F statistic	p-value	Eta-squared	Interpretation
All models including YOLOv8n-DD	84.963	2.098e-41	0.964	Significant at $p < 0.05$
All models excluding YOLOv8n-DD	2.528	0.002242	0.446	Significant at $p < 0.05$

Model A	Model B	Mean difference	p-value	Significant
YOLOv5m	YOLOv10s	0.033	0.0273	Yes
YOLOv5m	YOLOv10n	0.032	0.0386	Yes

The competitive-model ANOVA excluding YOLOv8n-DefectDef gives $p = 0.0022$ and eta-squared = 0.446. This indicates statistically significant differences among the competitive model set and a meaningful model effect on AP50 variation. Tukey HSD identifies YOLOv5m as significantly better than YOLOv10s and YOLOv10n under the benchmark protocol.

5. Discussion

The results support the idea that model selection for surface defect detection should not depend on a single reported AP50 score. YOLOv5m has the highest mean AP50, but the top models are close enough that a simple ranking could exaggerate practical differences. The statistical testing step helps distinguish between small numerical differences and differences that are supported across folds.

The split sensitivity result is especially important from an industrial engineering perspective. In production, inspection data may be limited, imbalanced, or affected by changes in lighting, material, surface texture, and defect frequency. If model performance changes when the train/test split changes, then deploying a model based on one

favorable split could create quality risk. A missed defect can allow a nonconforming component to continue downstream, while a false positive can increase rework, scrap, or manual review workload.

The mini app included in the repository communicates this point visually. It is not live YOLO inference; instead, it illustrates how model choice, fold choice, and split choice affect benchmark interpretation. The app frames the model as a decision-support tool for quality engineering rather than a fully autonomous production decision system.

Overall, the most defensible conclusion is that YOLOv5m is the strongest candidate in the provided benchmark table, but deployment would still require validation on real factory images. The project demonstrates a reproducible comparison workflow rather than claiming that the stored benchmark results alone are sufficient for production deployment.

6. Evaluation and Reflection

The project successfully turns the selected research paper and project proposal into a reproducible artifact. The repository can regenerate the main summary tables and plots, run statistical testing, and provide a small dashboard for explaining the benchmark logic. This supports transparent evaluation and makes the project easier to reproduce or extend.

A major limitation is that the default repository path does not include the NEU image dataset, trained YOLO weights, or full model training logs. This is intentional because the dataset and weights are external and can be large. As a result, the default analysis verifies the benchmark tables and statistical calculations rather than retraining all models locally. Full experimental replication would require downloading the NEU dataset, preparing annotations, installing Ultralytics, and running fold-level model training or validation with suitable compute resources.

Another limitation is dataset generalization. NEU is a controlled public benchmark with 1,800 grayscale images. Real production settings may include different materials, camera angles, illumination, surface finishes, defect frequencies, and labeling practices. Thus, a NEU good-performing model should be a prototype candidate, not a production solution guaranteed.

From both an ethical and operational perspective, our work reinforces that automated inspection should be used as an aid to human quality decision-making. Missed defects can affect product safety and reliability, and false positives can increase operating expense. Human oversight, continuous validation, and monitoring of performance drift are still needed before this approach can be used in actual manufacturing settings.

7. Artifacts and Reproducibility

The project artifacts are organized in the GitHub repository and local zip package. The repository contains the analysis package, AP50 CSV data, generated figures, statistical summaries, optional YOLO scripts, unit tests, and a static mini app.

Artifact	Purpose
surface_defect_benchmark Python package	Loads AP50 tables, validates inputs, ranks models, runs ANOVA/Tukey HSD, and generates plots.
data/*.csv	Stores split sensitivity, fold AP50, model AP50, and p-value inputs used for reproducible analysis.
reports/summary/*.csv	Contains generated model rankings, ANOVA summary, and Tukey HSD pairwise comparisons.
reports/figures/*.png	Contains four generated figures for the Results section.
mini_app/index.html	Interactive browser dashboard illustrating benchmark decisions and inspection cases.

scripts/prepare_neu.py	Optional NEU annotation conversion and fold preparation script.
scripts/train_yolo_folds.py and evaluate_yolo_folds.py	Optional training and run aggregation scripts for full YOLO experiments.
tests/	Automated tests for data loading, statistics, annotation conversion, and fold creation.

Repository link: https://github.com/dannotquan/automated-surface-defect-identification-in-production

Local mini app path: surface-defect-benchmark/mini_app/index.html

Local zip artifact: surface-defect-benchmark.zip

8. References

- Lema, D. G., Sanchez-Gonzalez, L., Usamentiaga, R., & delaCalle, F. J. (2025). Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach. Journal of Intelligent Manufacturing. https://doi.org/10.1007/s10845-025-02672-8
- Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. Applied Surface Science, 285, 858-864. https://doi.org/10.1016/j.apsusc.2013.09.002
- Ultralytics. YOLO object detection framework. https://github.com/ultralytics/ultralytics
- NEU surface defect database. Kaggle dataset. https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database